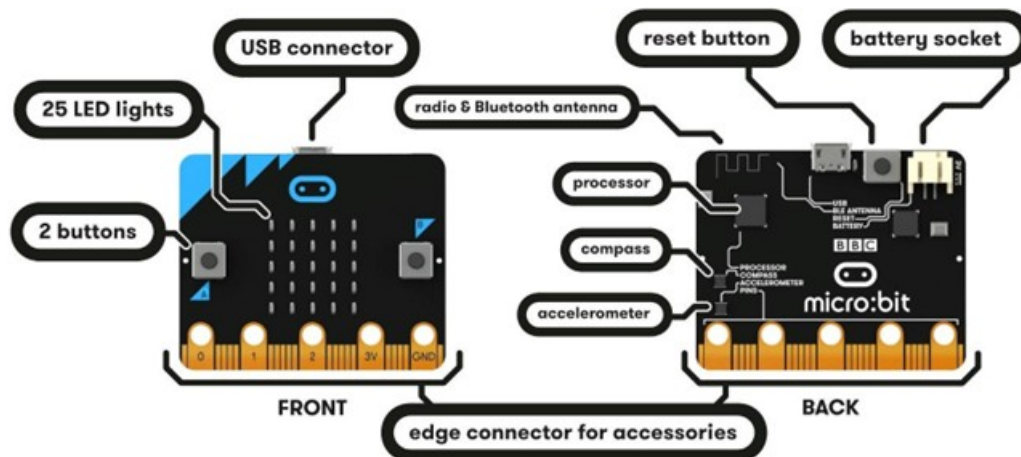


Conocer el Hardware

Empecemos por el principio, el hardware. A continuación podemos ver una representación de la placa con la que vamos a trabajar en este manual.



La micro:bit tiene las siguientes características hardware:

- 25 LED programables individuales
- 2 botones programables
- Pines de conexión física
- Sensores de luz y temperatura
- Sensores de movimiento (acelerómetro y brújula)
- Comunicación inalámbrica, vía radio y Bluetooth
- Interfaz USB

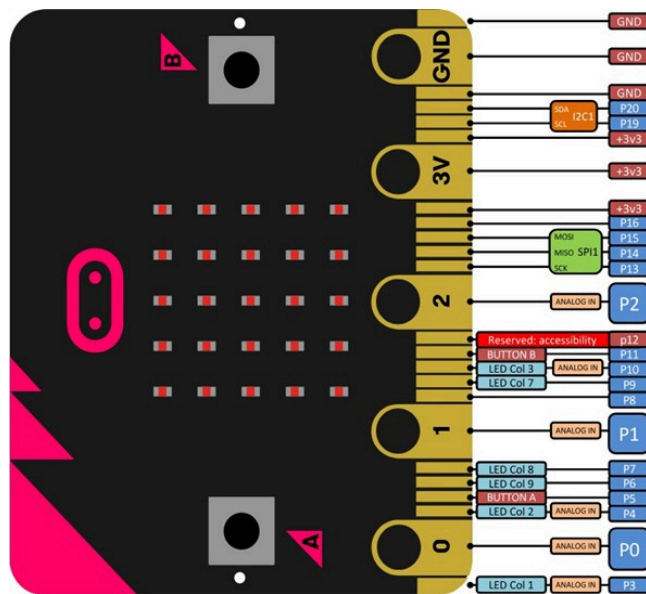
Para más información sobre la micro:bit, podéis visitar la página:

<https://microbit.org/guide/features/>.

No es necesario que los principiantes dominen esta sección, pero sí es necesario tener una idea general. Sin embargo, si queremos ser desarrolladores, la información sobre el hardware resultará muy útil. Encontraremos información detallada sobre el hardware de la micro:bit [aquí](#).

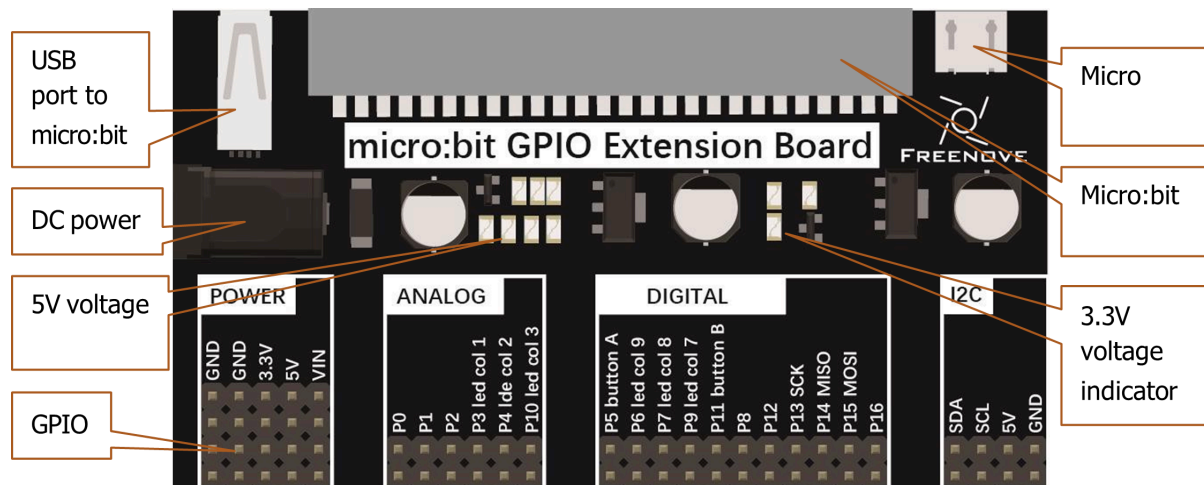
GPIO

La GPIO, es decir, los pines de entrada/salida de uso general, son un componente importante de la micro:bit para conectar dispositivos externos. Todos los sensores y dispositivos de la placa se comunican entre sí a través de los GPIO del micro:bit. A continuación se muestra el diagrama de numeración y funciones:



GPIO - Placa de extensión

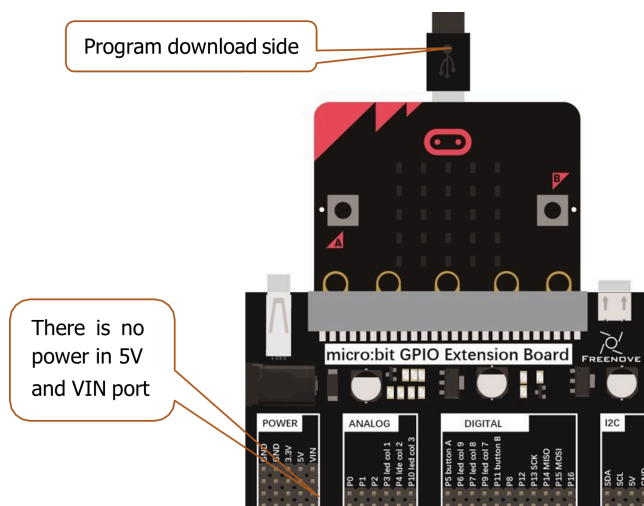
La placa de extensión es un accesorio que permite conectar la micro:bit a otros dispositivos electrónicos conectado a la placa micro:bit a través del socket correspondiente. Esta placa proporciona una forma más fácil de acceder a los pines GPIO de la micro:bit, lo que facilita la conexión de sensores, actuadores y otros componentes electrónicos. A continuación se muestra un ejemplo de una placa de extensión:



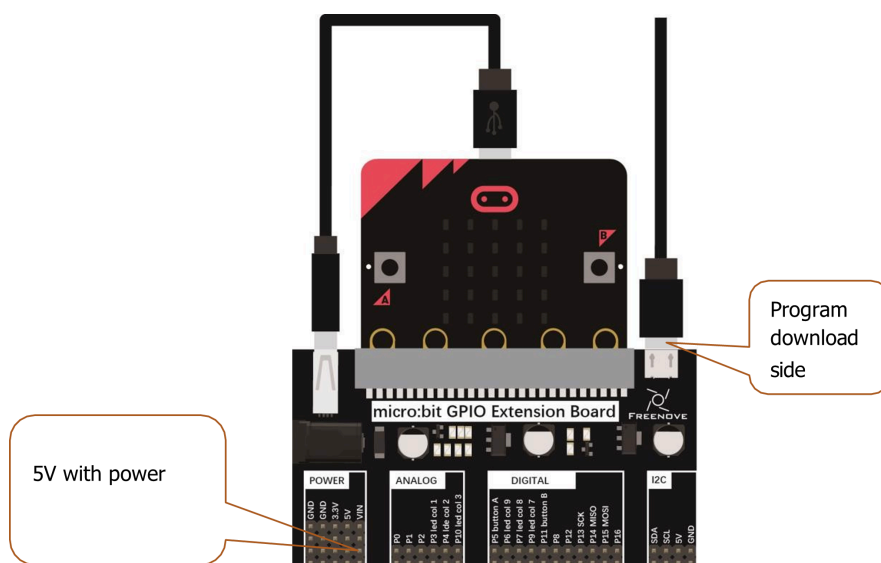
En esta extensión se han añadido puertos de entrada - salida adicionales con voltajes de 5V y VIN(9V) para suplir los requerimientos de algunos dispositivos.

Cómo usar la extensión

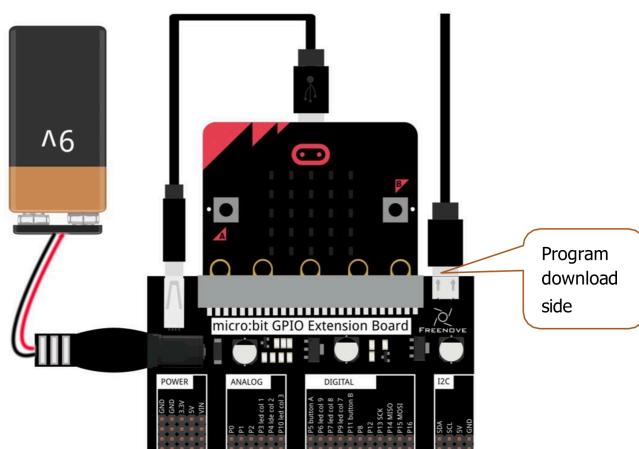
Si no es necesario utilizar los voltajes extendidos podemos hacer uso de la placa de la siguiente manera:



Si los periféricos necesitan 5V, pero no mucha potencia, la siguiente imagen muestra cómo hacerlo:



Por último, si las necesidades son elevadas, el diagrama siguiente muestra la configuración adecuada:



Programación de la Micro::bit 2

Con este manual vamos a aprender a programar la Micro::bit 2, un microcontrolador diseñado para la educación y la experimentación en electrónica y programación. La Micro::bit 2 cuenta con una variedad de sensores, botones, una pantalla LED y conectividad Bluetooth, lo que la hace ideal para proyectos interactivos. Utilizaremos como base el lenguaje Rust, que es un lenguaje de programación moderno y seguro, para crear programas que interactúen con los componentes de la Micro::bit 2.

Referencias

Antes de nada unas pocas referencias:

- [Documentación oficial de la Micro::bit 2](#)
- [Documentación de Rust](#)
- [micro::bit v2 Embedded Discovery Book \(castellano\)](#)
- [Rust Embedded Book](#)

Instalación del entorno de desarrollo

Para programar la Micro::bit 2 con Rust, necesitamos configurar nuestro entorno de desarrollo. Antes de pasar a la instalación real, enumeraremos los requisitos que he probado en este manual:

- Rust 1.79.0 o una toolchain más reciente.
- `gdb-multiarch`. Herramienta de depuración. La versión más antigua que hemos probado es la 10.2, pero es probable que otras versiones también funcionen. Si la distribución/plataforma no tiene `gdb-multiarch` disponible, podemos usar `arm-none-eabi-gdb` para depurar. Además, algunos binarios de `gdb` están contruidos con capacidades multiplataforma: se puede encontrar más información sobre esto en el capítulo de depuración de este libro.
- `[ cargo-binutils ]`. Versión 0.3.6 o posterior. `[ cargo-binutils ]`: <https://github.com/rust-embedded/cargo-binutils>
- `[ probe-rs-tools ]`. Versión 0.24.0 o más reciente. `[ probe-rs-tools ]`: <https://probe.rs/docs/overview/about-probe-rs/>
- `minicom` en Linux y macOS. Se ha probado la versión: 2.7.1. Esperamos que versiones posteriores también funcionen.
- `PuTTY` en Windows.

Instalación de Rust

Para instalar Rust, podemos utilizar `rustup`, que es la herramienta oficial para gestionar las versiones de Rust. Si no tenemos `rustup` instalado, seguiremos las instrucciones en la [página oficial de Rust](#).

Instrucciones de la página: Para empezar a usar Rust, descarga el instalador, ejecuta el programa y sigue las instrucciones que aparecen en pantalla. Es posible que tengas que instalar las herramientas de compilación de Visual Studio C++ cuando se solicite. Si no utilizas Windows, consulta "Otros métodos de instalación".

```
$ rustc -V
rustc 1.94.0 (4a4ef493e 2026-03-02)
```

Una vez instalado `rustup`, tenemos que instalar la toolchain de Rust con el comando:

```
$ rustup target add thumbv7em-none-eabihf
```

Solo hay que hacerlo una vez; `rustup` actualizará automáticamente la cadena de compilación si se lo pedimos. Explicaremos más adelante por qué es necesario instalar esta toolchain específica.

Instalación bajo Windows

La mayoría de ordenadores educativos utilizan Windows, por lo que vamos a detallar los pasos para instalar Rust en este sistema operativo. Si estamos usando otra plataforma recomiendo leer el libro de micro::bit v2 Embedded Discovery Book.

`gdb-multiarch (arm-none-eabi-gdb)`

Hemos descargado e instalado el siguiente archivo de la página oficial de Arm: [Aquí](#)

Arm proporciona ejecutables (`.exe`) para Windows. Se pueden encontrar en [gcc](#), solo hay que seguir las instrucciones. Justo antes de que finalice el proceso de instalación, hay que marcar/seleccionar la opción “Añadir ruta a la variable de entorno”. Si no aparece dicha opción se tendrá que hacer desde el diálogo del sistema y añadir la siguiente ruta para la instalación por defecto:

```
C:\Program Files\Arm\GNU Toolchain mingw-w64-x86_64-arm-none-eabi\bin
```

A continuación, comprobaremos que las herramientas se encuentran en el `%PATH%`:

```
$ arm-none-eabi-gcc -v
(..)
gcc version 15.2.1 20251203 (Arm GNU Toolchain 15.2.Rel1 (Build arm-15.86))
```

`cargo-binutils`

```
$ rustup component add llvm-tools
$ cargo install cargo-binutils --vers '^0.4'
$ cargo size --version
cargo-size 0.4.0
```

`probe-rs-tools`

NOTA Si existen en el sistema versiones anteriores de `probe-run`, `probe-rs` o `cargo-embed` instaladas, hay que eliminarlas antes de seguir adelante, ya que podrían causar problemas. En particular, `probe-run` ya no existe oficialmente. Hay que eliminarlas si es necesario:

```
$ cargo uninstall cargo-embed
$ cargo uninstall probe-run
$ cargo uninstall probe-rs
$ cargo uninstall probe-rs-cli
```

Para instalar `probe-rs-tools`, hay que seguir las instrucciones de la página oficial en <https://probe.rs>. En mi caso la instalación mediante cargo no me ha dado ningún tipo de problema.

```
$ cargo install probe-rs-tools
```

```
Finished `release` profile [optimized] target(s) in 3m 34s
Installing C:\Users\arcipreste\.cargo\bin\cargo-embed.exe
Installing C:\Users\arcipreste\.cargo\bin\cargo-flash.exe
Installing C:\Users\arcipreste\.cargo\bin\probe-rs.exe
Installed package `probe-rs-tools v0.31.0` (executables `cargo-embed.exe`, `cargo-
flash.exe`, `probe-rs.exe`)
```

```
C:\Users\...>probe-rs
The probe-rs CLI
```

Instalar la herramienta `probe-rs-tools` configurará en nuestro ordenador diversas herramientas muy útiles, incluyendo `probe-rs` y `cargo-embed` (que normalmente se ejecutan como un comando de Cargo). Debemos comprobar que todo funciona correctamente antes de pasar a la siguiente sección.

```
$ cargo embed --version
cargo embed 0.31.0 (git commit: crates.io)
```

PuTTY

Se puede descargar `putty.exe` desde [aquí](#) y añadirlo al `%PATH%`.

Primer ejemplo: “Hola, mundo”

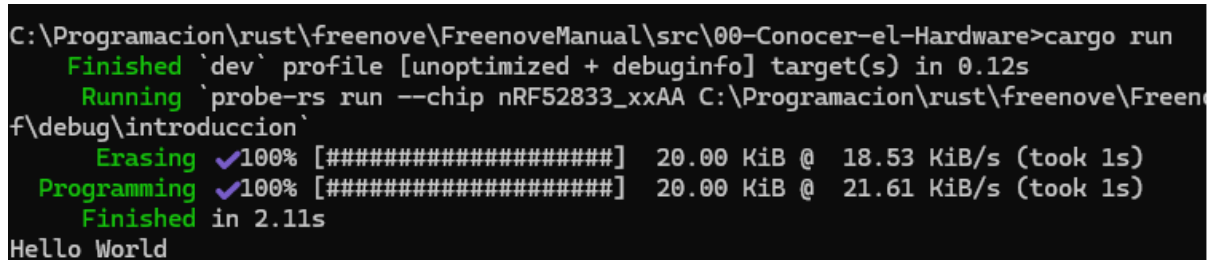
Vamos a crear nuestro primer programa para la Micro::bit 2, que mostrará el mensaje “Hola, mundo” en el terminal. Para ello, seguiremos los siguientes pasos:

- Conectamos la placa a un puerto USB de nuestro ordenador. Comprobamos que se abre una ventana con el contenido de la MB2.
- Abrimos un terminal y nos situamos en el directorio del capítulo inicial:

```
$ cd src/00-Conocer-el-Hardware/
```

- Ejecutamos el siguiente comando para compilar y ejecutar el programa:

```
$ cargo run
```



```
C:\Programacion\rust\freenove\FreenoveManual\src\00-Conocer-el-Hardware>cargo run
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.12s
Running `probe-rs run --chip nRF52833_xxAA C:\Programacion\rust\freenove\FreenoveManual\src\00-Conocer-el-Hardware\src\main.rs`
Erasing ✓100% [#####] 20.00 KiB @ 18.53 KiB/s (took 1s)
Programming ✓100% [#####] 20.00 KiB @ 21.61 KiB/s (took 1s)
Finished in 2.11s
Hello World
```

El código fuente del programa se encuentra en el archivo `src/main.rs`. A continuación, se muestra el contenido del archivo:

```
#![no_main]
#![no_std]

use cortex_m::asm::{nop};
use nrf52833_pac as _;
use panic_rtt_target as _;
use rtt_target::{rprintln, rtt_init_print};

use cortex_m_rt::entry;

#[entry]
fn main() -> ! {
    rtt_init_print!();
    rprintln!("Hello World");
    loop {
        nop();
    }
}
```

Un poco más de programación

Depuración de programas

La depuración de programas es una parte fundamental del desarrollo de software, especialmente cuando se trabaja con microcontroladores como la Micro::bit. Para un funcionamiento correcto de la depuración, necesitamos configurar el proyecto para que utilice `gdb` en lugar de `rtt`. Para ello, editamos el archivo `Embed.toml` y establecemos `enabled = false` para `rtt` y `enabled = true` para `gdb`. El contenido del archivo debe ser el siguiente:

```
[default.rtt]
enabled = false

[default.gdb]
enabled = true
```

Tras modificar el fichero `Embed.toml` en el directorio `00-Conocer-el-Sistema`, ejecutamos pero lanzando la depuración con `cargo embed`:

```
$ cargo embed
```

Nos conectamos a la sesión de depuración con `gdb` desde otra consola (recordar que solo se muestran ejemplos para el SSOO Windows). Para ello, abrimos otra terminal y nos situamos en la raíz del proyecto. Ejecutamos `arm-none-eabi-gdb` con el binario que queremos depurar como argumento:

```
$ cd raiz_del_libro
$ arm-none-eabi-gdb ./target/thumbv7em-none-eabihf/debug/introduccion
```

Si nos da un fallo donde no encuentra el binario, recordar que el directorio `target` se encuentra en la raíz del proyecto, no en `src/00-Conocer-el-Sistema`. Ya dentro de `gdb` nos unimos a la sesión de depuración remota con el comando `target remote` y el puerto que vemos al ejecutar con `cargo embed` (en este caso, el puerto 1337):

```
(gdb) target remote :1337
Remote debugging using :1337
```

Los puntos de ruptura se pueden usar para detener el flujo normal de un programa. El comando `continue` permitirá que el binario se ejecute libremente *hasta* que alcance un punto de ruptura. En este caso, hasta que alcance la función `main` porque hemos establecido uno allí.

```
(gdb) break main
...
(gdb) continue
...
```

Para ver el contenido de las variables, podemos usar el comando `print` seguido del nombre de la variable. Por ejemplo, para ver el contenido de la variable `x`:

```
(gdb) print x
$1 = 0
```

Si queremos ejecutar el programa paso a paso, podemos usar el comando `next` para avanzar una línea de código a la vez. Esto nos permite observar cómo cambian las variables y el flujo del programa en tiempo real.


```
(gdb) next
16         loop {}
```

El comando `monitor reset` resetea el microcontrolador y lo para en el punto de entrada.

```
(gdb) monitor reset
```

Ya hemos terminado la sesión de depuración. Podemos salir con el comando `quit`.

```
(gdb) quit
A debugging session is active.

        Inferior 1 [Remote target] will be detached.

Quit anyway? (y or n) y
Detaching from program: $PWD/target/thumbv7em-none-eabihf/debug/meet-your-software,
Remote target
Ending remote debugging.
[Inferior 1 (Remote target) detached]
```

Comprendiendo el primer programa

Vamos a echar un vistazo a nuestro primer programa. Comprobemos el fichero `src/main.rs`:

```
#![no_main]
#![no_std]

use cortex_m::asm::{nop};
use nrf52833_pac as _;
use panic_rtt_target as _;
use rtt_target::{rprintln, rtt_init_print};

use cortex_m_rt::entry;

#[entry]
fn main() -> ! {
    rtt_init_print!();
    rprintln!("Hello World");
    loop {
        nop();
    }
}
```

Los programas para microcontroladores son diferentes de los programas estándar en dos aspectos:

`#![no_std]` y `#![no_main]`.

El atributo `no_std` indica que este programa no usará la biblioteca estándar de Rust, la cual asume un sistema operativo subyacente; el programa utilizará en su lugar el crate `core`, un subconjunto de `std` que puede ejecutarse en sistemas hardware directamente.

El atributo `no_main` indica que este programa no usará la interfaz estándar 'main', que está diseñada para aplicaciones de línea de comandos que reciben argumentos. En lugar del 'main' estándar, usaremos el atributo 'entry' del crate `cortex-m-rt` para definir un punto de entrada personalizado. En este programa hemos definido el punto de entrada como `main`, pero se podría haber usado cualquier otro nombre. La función del punto de entrada debe tener la firma `fn() -> !`; este tipo indica que la función no termina. Significa que el programa nunca finaliza: si el compilador detecta que esto sería posible, se negará a compilarlo.

Si observamos con cuidado el directorio del proyecto notaremos que hay un directorio `.cargo` posiblemente oculto en el proyecto. Este directorio contiene un archivo de configuración de Cargo `.cargo/config.toml`.

```
[build]
target = "thumbv7em-none-eabi"

[target.thumbv7em-none-eabi]
runner = "probe-rs run --chip nRF52833_xxAA"
rustflags = [
    "-C", "linker=rust-ld",
]
```

Este fichero modifica el proceso de enlace para adaptar la disposición de memoria del programa a los requisitos del dispositivo de desarrollo. Este proceso de enlace es un requisito del crate `cortex-m-rt`. El archivo `.cargo/config.toml` también le dice a Cargo cómo construir y ejecutar el código en nuestra MB2.

Hay también un fichero `Embed.toml`:

```
[default.probe]
protocol = "Swd"

[default.general]
chip = "nrf52833_xxAA"

[default.reset]
halt_afterwards = true

[default.rtt]
enabled = true

[default.gdb]
enabled = false
```

Este fichero informa a `cargo-embed` que:

- Trabaja con un chip NRF52833.
- Queremos detener la ejecución en el chip después de flashearlos, por lo que nuestro programa se detiene antes de `main`.
- Queremos deshabilitar/habilitar RTT. RTT es un protocolo que permite al chip enviar texto a un depurador. Ya has visto RTT en acción fue la primera versión del programa inicial.
- Queremos Deshabilitar/habilitar GDB. Este paso es necesario para procesos de depuración tal y como hemos presentado al comienzo de este capítulo.

Generando el binario

El primer paso es construir el binario. Dado que el microcontrolador tiene una arquitectura diferente a la de nuestro ordenador, tendremos que realizar una compilación cruzada. Hacerlo en Rust es tan simple como pasar un flag extra `--target` a `rustc` o Cargo. La parte complicada es averiguar el argumento de ese flag: el *nombre* del destino.

Como ya hemos visto, el microcontrolador del micro:bit tiene un procesador Cortex-M4F. `rustc` sabe cómo compilar de forma cruzada para la arquitectura Cortex-M y proporciona varios destinos que cubren las diferentes familias de procesadores dentro de esa arquitectura:

- `thumbv6m-none-eabi`, para los procesadores Cortex-M0 y Cortex-M1
- `thumbv7m-none-eabi`, para el procesador Cortex-M3
- `thumbv7em-none-eabi`, para los procesadores Cortex-M4 y Cortex-M7
- `thumbv7em-none-eabihf`, para los procesadores Cortex-M4F y Cortex-M7F
- `thumbv8m.main-none-eabi`, para los procesadores Cortex-M33 y Cortex-M35P
- `thumbv8m.main-none-eabihf`, para los procesadores Cortex-M33F y Cortex-M35PF

“Thumb” aquí se refiere a una versión del conjunto de instrucciones Arm que tiene instrucciones más pequeñas para reducir el tamaño del código. La denominación `hf / F` significa que implementa aceleración de punto flotante por hardware.

Para la MB2, micro:bit v2, queremos el destino `thumbv7em-none-eabihf`.

Antes de realizar la compilación cruzada, es necesario descargar una versión precompilada de la biblioteca estándar (en realidad, una versión reducida de ella) en el host local. Se hace usando `rustup`:

```
$ rustup target add thumbv7em-none-eabihf
```

Solo hay que hacerlo una vez; `rustup` actualizará este destino (reinstalando un nuevo componente de la biblioteca estándar `rust-std` que contiene la biblioteca `core` que usamos) cada vez que actualicemos la cadena de compilación. Por lo tanto, se puede omitir este paso si ya se agregó la toolchain necesaria anteriormente.

Con el componente `rust-std` en su lugar, ya es posible compilar el programa de forma cruzada usando Cargo. Nos aseguraremos de estar en el directorio `src/00-Conocer-el-Sistema`, luego lo construimos. Este código inicial es un ejemplo, así que lo compilamos como tal.

```
$ cargo build
   Compiling semver-parser v0.7.0
   Compiling proc-macro2 v1.0.86
   ...

Finished dev [unoptimized + debuginfo] target(s) in 33.67s
```

Ok, ya tenemos un ejecutable. No hará mucho, imprime el mensaje “Hello, World!”.

Flashearlo

Flashear es el proceso de mover el programa a la memoria del microcontrolador. Una vez hecho, el microcontrolador ejecutará el programa cada vez que se encienda.

El programa será el único existente en la memoria. Por esto me refiero a que no hay nada más ejecutándose en el microcontrolador: ni un sistema operativo, ni un “daemon”, nada. Nuestro programa tiene control total sobre el dispositivo.

Pasarlo al microcontrolador es muy simple, gracias a `cargo embed\run`.

```
$ cargo run
(...)
Erasing sectors ✓ [00:00:00]
[#####] 2.00KiB/ 2.00KiB @
4.21KiB/s (eta 0s )
Programming pages ✓ [00:00:00]
[#####] 2.00KiB/ 2.00KiB @
2.71KiB/s (eta 0s )
Finished flashing in 0.608s
```

Es importante fijarse que si queremos depurar se tiene que lanzar con `cargo embed`, y de esta forma no termina después de mostrar la última línea. Esto es intencionado: no hay que cerrar `cargo embed`, ya que lo necesitamos en este estado para depurar.